

Министерство образования Республики Беларусь
Учреждение образования
«Белорусский государственный университет информатики и
радиоэлектроники»

Кафедра электронных вычислительных машин

Лабораторная работа №4
« Программирование часов реального времени»

Выполнил:
Студент группы 150503
Шарай П.Ю.

Проверил:
к. т. н., доцент
Одинец Д.Н.

Минск 2023

1. Постановка задачи

Задание по лабораторной работе состоит из двух частей. Первая часть общая для всех,

вторая – по вариантам, выдаваемым преподавателем.

Первая часть (общее задание). Написать программу, которая будет считывать и

устанавливать время в часах реального времени. Считанное время должно выводиться на

экран в удобочитаемой форме.

Условия выполнения лабораторной работы для первой части:

1 Перед началом установки новых значений времени необходимо считывать и

анализировать старший байт регистра состояния 1 на предмет доступности значений для

чтения и записи. Начинать операцию записи новых значений, можно только в случае, когда

этот бит установлен в '0' – то есть, регистры часов доступны.

2 После проверки доступности регистров (пункт 1), необходимо отключить

внутренний цикл обновления часов реального времени, воспользовавшись старшим битом

регистра состояния 2 После окончания операции установки значений этот бит должен быть

сброшен, для возобновления внутреннего цикла обновления часов реального времени.

3 Новые значения для регистров часов реального времени должны вводиться с

клавиатуры в удобном для пользователя виде.

Вторая часть.

1 Используя аппаратное прерывание часов реального времени и режим генерации

периодических прерываний реализовать функцию задержки с точностью в миллисекунды (с

возможностью изменения частоты генерации).

Условия выполнения данного варианта:

1.1.

Задержка должна вводиться с клавиатуры в миллисекундах в удобной для

пользователя форме.

1.2.

После окончания периода задержки программа должна сообщить об этом в любой

форме. При этом зависание компьютера не допускается.

1.3.

Предусмотреть возможность изменения частоты генерации периодических

прерываний вводом с клавиатуры соответствующих значений

2 Используя аппаратное прерывания часов реального времени и режим будильника

ЧРВ (4А использовать не желательно) реализовать функции программируемого будильника.

2.1.

Время будильника вводится с клавиатуры в удобной для пользователя форме.

2.2.

При срабатывания будильника программа должна сообщить об этом в любой

форме. При этом зависание компьютера не допускается.

Общие условия для всей лабораторной работы.

1 В программе должно быть реализовано меню, позволяющее выбрать тестируемый

функционал (установка времени, считывание времени, задержка и т.д.)

2 Весь ввод/вывод данных с/на консоль должен выполняться в удобной для

пользователя форме.

3 Зависание компьютера не допускается ни в ходе работы программы, ни после ее завершения.

2. Алгоритм решения

Считывание и установка времени:

- Считываем и анализируем старший байт регистра состояния 1 на предмет доступности значений для чтения и записи. Начинать операцию записи новых значений, можно только в случае, когда этот бит установлен в '0' – то есть, регистры часов доступны.

- Отключаем внутренний цикл обновления часов реального времени, воспользовавшись старшим битом регистра состояния 2. После окончания операции установки значений этот бит должен быть сброшен, для возобновления внутреннего цикла обновления часов реального времени.

- Новые значения для регистров часов вводятся с клавиатуры.

3. Листинг программы

```
#include <dos.h>
#include <stdio.h>
#include <stdlib.h>
#include <conio.h>
```

```

#include <io.h>
#include <windows.h>

unsigned int delayTime = 0;
unsigned int delayMs;
unsigned int date[6];
unsigned int dateReg[] = { 0x00, 0x02, 0x04, 0x07, 0x08, 0x09 };

void interrupt(*oldDelay)(...);
void interrupt(*oldAlarm) (...);

void getTime();
void setTime();
void delay();
void setAlarm();
void inputTime();
void changeFrequency();
unsigned int decToBcd(unsigned int dec);
unsigned int bcdToDec(unsigned int bcd);

void interrupt newDelay(...)    // новый обработчик прерываний задержки
{
    delayTime++;
    outp(0x70, 0x0C);
    inp(0x71);

    outp(0x20, 0x20);          // сброс бита регистра обслуживаемых запросов на
    прерывание для master контроллера
    outp(0xA0, 0x20);          // сброс бита регистра обслуживаемых запросов на
    прерывание для slave контроллера
    if (delayTime == delayMs)
    {
        puts("Delay's end");
        disable();
        setvect(0x70, oldDelay); // возвращаем старый обработчик
        enable();
        outp(0x70, 0x0B);        // выбираем регистр для работы 0x0B
        outp(0x71, inp(0x71) & 0xBF); // 1011 1111 - сбрасываем периодические
        прерывания
    }
}

void interrupt newAlarm(...)    // новый обработчик прерывания будильника
{
    puts("Alarm!!!");

    outp(0x70, 0x0C);          // обязательно нужно обращаться к 0x71 после 0x70
    иначе поведение RTC непредсказуемо
    inp(0x71);
}

```

```

    outp(0x20, 0x20);          // сброс бита регистра обслуживаемых запросов на
прерывание для master контроллера
    outp(0xA0, 0x20);          // сброс бита регистра обслуживаемых запросов на
прерывание для slave контроллера

    disable();                 // запрещаем прерывания
    setvect(0x70, oldAlarm);    // возвращаем старый обработчик прерываний
    enable();                   // разрешаем прерывания
    outp(0x70, 0x0B);
    outp(0x71, inp(0x71) & 0xDF); // 1101 1111 - сбрасываем прерывания будильника
}

```

```

int main()
{
    while (1)
    {
        printf("1 - Time\n");
        printf("2 - Set time\n");
        printf("3 - Set alarm\n");
        printf("4 - Set delay\n");
        printf("5 - Set freq\n");
        printf("0 - Exit\n\n");
        switch (getch())
        {
            case '1':
                getTime();
                break;
            case '2':
                setTime();
                break;
            case '3':
                setAlarm();
                break;
            case '4':
                fflush(stdin);
                printf("Input delay (ms): ");
                scanf("%u", &delayMs);
                delay();
                break;
            case '5':
                changeFrequency();
                break;
            case '0':
                printf("\n\n");
                return 0;
            default:
                printf("\n\n");
                break;
        }
    }
}

```

```

}

void getTime()                // функция получения текущего времени
{
    unsigned char state;
    printf("\n\n");
    int i = 0;
    for (i = 0; i < 6; i++)
    {
        outp(0x70, 0x0A);      // выбор регистра 0x70
        state = inp(0x71);
        if (state >> 7)        // если часы заняты обновлением, то повторить итерацию
            заново
            {
                i--;
                continue;
            }
        outp(0x70, dateReg[i]); // выбор соответствующего регистра с еденицами
        времени
        date[i] = inp(0x71);
        date[i] = bcdToDec(date[i]); // перевод из bcd в десятичную сс и запись
    }

    printf("Date:\n%02u.%02u.%02u", date[2], date[1], date[0]);
    printf(" %02u.%02u.20%02u\n\n", date[3], date[4], date[5]);
}

void setTime()                // функция установки времени
{
    inputTime();
    disable();

    outp(0x70, 0x0B);
    outp(0x71, inp(0x71) | 0x80); // 1000 0000 - запрещаем обновление часов

    for (int i = 0; i < 3; i++) // цикл записи даты в регистры из dataReg введенного
        времени
        {
            outp(0x70, dateReg[i]);
            outp(0x71, date[i]);
        }

    outp(0x70, 0x0B);
    outp(0x71, inp(0x71) & 0x7F); // 0111 1111 - разрешаем обновление часов,
    остальные опции

    enable();                  // разрешение прерываний
    printf("\n\n");
}

void inputTime()              // ввод времени с проверкой формата, перевод в bcd

```

```

{
    int n;
    do {
        fflush(stdin);
        printf("Hours: ");
    } while ((scanf("%d", &n) != 1 || n > 23 || n < 0));
    date[2] = decToBcd(n);

    do {
        fflush(stdin);
        printf("Minutes: ");
    } while (scanf("%d", &n) != 1 || n > 59 || n < 0);
    date[1] = decToBcd(n);

    do {
        fflush(stdin);
        printf("Seconds: ");
    } while (scanf("%d", &n) != 1 || n > 59 || n < 0);
    date[0] = decToBcd(n);
}

void delay()                // функция для вызова задержки
{
    delayTime = 0;

    disable();              // cli

    oldDelay = getvect(0x70);    // сохраняем старый обработчик и меняем его на
    НОВЫЙ                      //
    setvect(0x70, newDelay);

    // размаскировка линии сигнала запроса от ЧРВ
    // 0xA1 - новое значение счетчика для системного таймера
    outp(0xA1, inp(0xA1) & 0xFE);    // 0xFE = 1111 1110
    // 0-й бит в 0 для разрешения прерывания от ЧРВ

    enable();               // sti

    outp(0x70, 0x0B);
    outp(0x71, inp(0x71) | 0x40);    // 0x40 == 0100 0000 - разрешение периодических
    прерываний

    return;
}

void setAlarm()
{
    unsigned int alarmReg[] = { 0x01, 0x03, 0x05 };

    inputTime();

```

```

disable();

oldAlarm = getvect(0x70);          // сохранение старого и установка нового векторов
прерываний
setvect(0x70, newAlarm);
outp(0xA1, (inp(0xA0) & 0xFE));

do
{
    outp(0x70, 0x0A);
} while (inp(0x71) >> 7); // цикл пока часы заняты

for (int i = 0; i < 3; i++)
{
    outp(0x70, alarmReg[i]);
    outp(0x71, date[i]); // установка даты в регистры из alarmReg
}

enable();

outp(0x70, 0x0B);
outp(0x71, inp(0x71) | 0x20); //0010 0000 - разрешаем прерывание будильника

printf("Alarm enable\n\n");
}

void changeFrequency()
{
    int freq;
    int q;
    int bin;
    puts("set freq:");
    puts("1 - 2 hertz");
    puts("2 - 4 hertz");
    puts("3 - 8 hertz");
    puts("4 - 16 hertz");
    puts("5 - 32 hertz");
    puts("6 - 64 hertz");
    puts("7 - 128 hertz");
    puts("8 - 256 hertz");
    puts("9 - 512 hertz");
    puts("10 - 1024 hertz");
    puts("11 - 2048 hertz");
    puts("12 - 4096 hertz");
    puts("13 - 8192 hertz");
    scanf("%d", &q);
    freq = 0x0F - q + 1;

    outp(0x70, 0x0A);
    bin = inp(0x71);

```



```

printf("before =%X= \n", bin); // 1024 hz == 0x26 == 0010 0110

outp(0x70, 0x0A);
outp(0x71, inp(0x71) | 0x0F); // 0010 1111 - частота периодического прерывания
равна 500 мс(младшие 4 бита), делитель фазы 32768hz по умолчанию

outp(0x70, 0x0A);
outp(0x71, inp(0x71) & (0xA0 + freq)); // 0010 xxxx - установка новой частоты

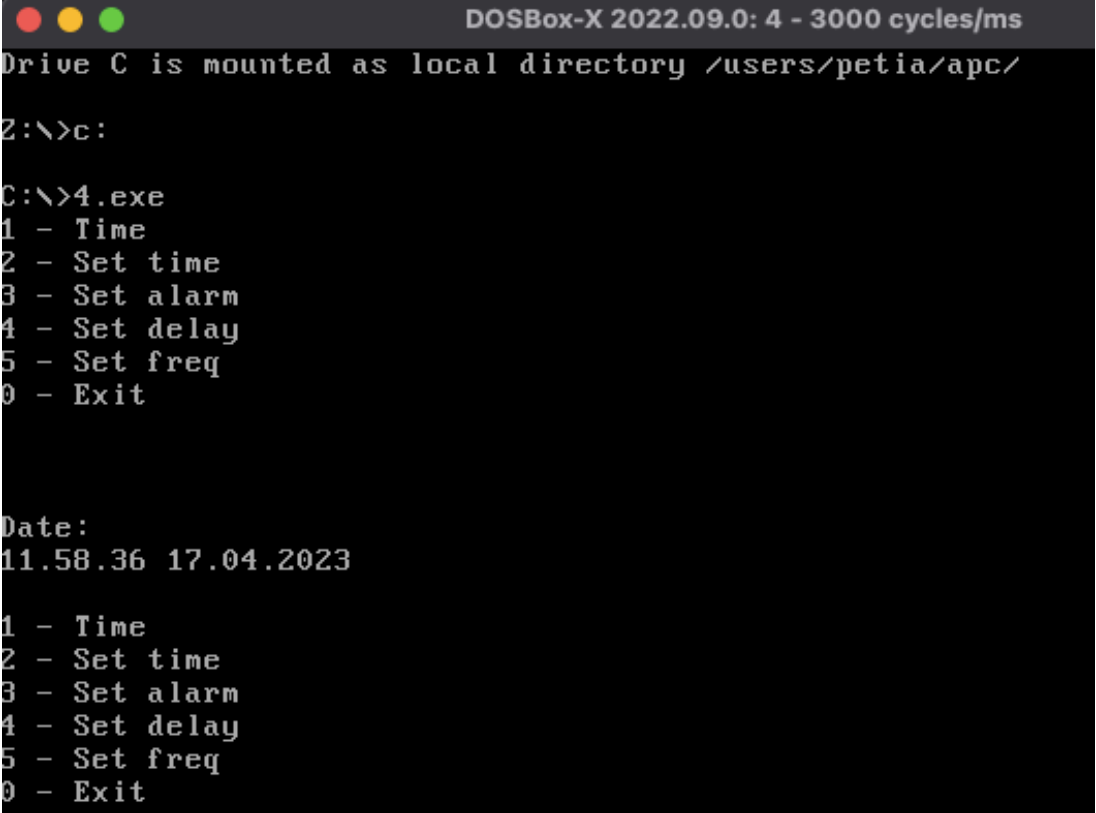
outp(0x70, 0x0A);
bin = inp(0x71); // новая частота

printf("\nafter =%X= \n", bin);
}

unsigned int bcdToDec(unsigned int bcd)
{
    return (bcd / 16 * 10) + (bcd % 16);
}
unsigned int decToBcd(unsigned int dec)
{
    return (dec / 10 * 16) + (dec % 10);
}

```

4. Тестирование



```

DOSBox-X 2022.09.0: 4 - 3000 cycles/ms
Drive C is mounted as local directory /users/petia/apc/
Z:\>c:
C:\>4.exe
1 - Time
2 - Set time
3 - Set alarm
4 - Set delay
5 - Set freq
0 - Exit

Date:
11.58.36 17.04.2023

1 - Time
2 - Set time
3 - Set alarm
4 - Set delay
5 - Set freq
0 - Exit

```

Рисунок 4.1 – Демонстрация времени

5. Заключение

В ходе лабораторной работы удалось вывести текущее время, установить новое время, проверить срабатывание задержки и будильника.